

MODULE 3: OPERATING SYSTEMS PRINCIPLES
WEEK 3

Learning Outcomes:

After completing this module you are expected to understand the following:

- 1. The evolution of hardware/software techniques and application needs
- 2. Device organization
- 3. Interrupts: methods and implementations
- 4. Concept of user/system state and protection, transition to kernel mode

A. Engage

Quotation Station

	SOFTWARE
	HARDWARE
	 EXTREME HARDWARE

Figure 3.0: Software and Hardware.

B. Explain

Computer **hardware** is any physical device used in or with your machine, whereas **software** is a collection of codes installed onto your computer's hard drive. For instance, the computer monitors you're currently using to read this text and the mouse you're using to navigate this web page is computer **hardware**.

ASIAN INSTITUTE OF COMPUTER STUDIES

Bachelor of Science in Computer Science

Course Modules

CS4114 - Operating System

4th Year – 1st Semester

C. Explore

Video Titles:

1. TYPES OF SOFTWARE, APPLICATION SOFTWARE, SYSTEM SOFTWARE, COMPUTER BASICS.
2. Computer Science Basics: Hardware and Software.

YouTube Links:

1. <https://www.youtube.com/watch?v=BTB86HeZVwk>
2. https://www.youtube.com/watch?v=vG_qmtdBPTU

Module Video Filenames:

1. WEEK3_TYPES OF SOFTWARE.mp4
2. WEEK03_Computer Science Basics - Hardware and Software.mp4

D. Elaborate

Introduction

1. The evolution of hardware/software techniques and application needs

In order for a computer to function, it requires some kind of computer program or software. The computer hardware provides the capability to perform computing-related tasks. The software is the set of instructions that humans create to tell the computer what to do. Modern computers and software are two different concepts, but that wasn't always as apparent as it is now. The first mechanical computers, starting with Charles Babbage's Difference Engine in 1822, were entirely hardware-based and were programmed with switches and wires. Until the late 1940s, hardware and software were essentially the same things, and to reprogram a computer meant to rewire it, either partially or entirely. Computers themselves were not always ubiquitous like they are now. Originally, they were very difficult to use, very large, and too expensive for anyone except governments and universities to own.

Alan Turing developed the first theory for computer software in an essay he wrote in 1935, titled 'Computable Numbers With an Application to the Entscheidungsproblem'. The first software algorithm, however, was written for Babbage's Analytical Engine in 1842 by the Countess of Lovelace, Augusta Ada King-Noel, also known as Ada Lovelace. The goal of her algorithm, which existed as a set of notes, was to direct the Analytical Engine to calculate Bernoulli numbers. Lovelace also speculated that the Analytical Engine could perform tasks beyond the intent of its design. She stated in her notes that it could carry out more general computing functions such as composing music.

All computer software depends on and is created with some form of programming language. Early programming languages were machine-dependent. These machine languages took significant levels of expertise and training and made software development highly expensive. New languages developed that were human-readable and easier to manage, such as FORTRAN in 1957 and COBOL in 1959. These languages required a compiler to translate them into machine-readable code and were designed to develop software for the military, scientific community, and businesses. When personal computers entered the market, simpler, interpreted languages such as BASIC made software coding a task that even home computer users could master. Interpreted languages did not need a compiler to run and were very easy to debug and quick to run. Another important language was LOGO, developed in 1967 to help children get involved in programming. One of the most prevalent languages today is C or C++, from which most modern software is made.

Computers and software are still evolving. Scientists are currently working on quantum computers that use quantum bits, or qubits, instead of binary digits. These computers may be capable of running millions of times faster than computers of today. Software design for this emerging class of computer is still in its infancy, and it relies on principles of quantum physics that are difficult to explain to the masses. Companies like IBM and Microsoft, however, are already working to make quantum computer software development possible for mainstream users.

2. Interrupts: methods and implementations

Interrupts are signals sent to the CPU by external devices, normally I/O devices. They tell the CPU to stop its current activities and execute the appropriate part of the operating system.

There are three types of interrupts:

Hardware Interrupts are generated by hardware devices to signal that they need some attention from the OS. They may have just received some data (e.g., keystrokes on the keyboard or an data on the ethernet card); or they have just completed a task which the operating system previous requested, such as transferring data between the hard drive and memory.

Software Interrupts are generated by programs when they want to request a system call to be performed by the operating system.

Traps are generated by the CPU itself to indicate that some error or condition occurred for which assistance from the operating system is needed.

Interrupts are important because they give the user better control over the computer. Without interrupts, a user may have to wait for a given application to have a higher priority over the CPU to be ran. This ensures that the CPU will deal with the process immediately.

There are two modes of execution, known as user mode and kernel or supervisor mode. User mode is restricted in that certain instructions cannot be executed, certain registers cannot be accessed, and I/O devices can not be accessed. Kernel mode has none of these restrictions. A system call will set the CPU to kernel mode, as will traps and interrupts. Application programs cannot do this.

Mode bit: **Supervisor** or **User** mode

- *Supervisor mode*
 - Can execute all machine instructions
 - Can reference all memory locations
- *User mode*
 - Can only execute a subset of instructions
 - Can only reference a subset of memory locations

3. Concept of user/system state and protection, transition to kernel mode.

A key point towards understanding how operating systems work is to understand what the CPU does when an interrupt occurs. The hardware of the CPU does the exact same thing for each interrupt, which is what allows operating systems to take control away from the current running user process. The switching of running processes to execute code from the OS kernel is called a context switch.

CPUs rely on the data contained in a couple registers to correctly handle interrupts. One register holds a pointer to the process control block of the current running process. This register is set each time a process is loaded into memory. The other register holds a pointer to a table containing pointers to the instructions in the OS kernel for interrupt handlers and system calls. The value in this register and contents of the table are set when the operating system is initialized at boot time.

The CPU performs the following actions in response to an interrupt:

Using the pointer to the current process control block, the state and all register values for the process are saved for use when the process is later restarted.

The CPU mode bit is switched to supervisory mode.

Using the pointer to the interrupt handler table and the interrupt vector, the location of the kernel code to execute is determined. The interrupt vector is the IRQ for hardware interrupts (read from an interrupt

controller register) and an argument to the interrupt assembly language instruction for software interrupts.

Processing is switched to the appropriate portion of the kernel.

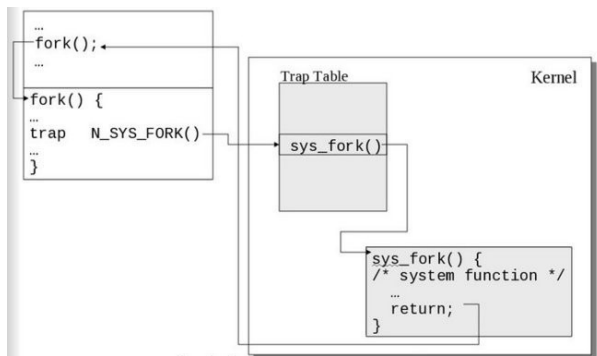


Figure 3.1. Kernel.

The CPU uses a table and the interrupt vector to find OS the code to execute in response to interrupts. A software interrupt is shown here.

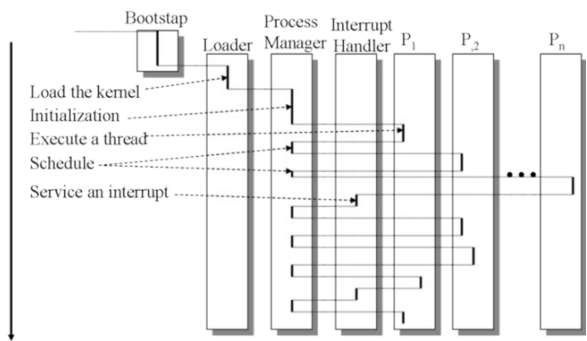


Figure. 3.2. Software.

As the computer runs, processing switches between user processes and the operating system as hardware and software interrupts are received.

E. Evaluate

ASSESSMENT:

Instructions: You may write your answer on the Answer Sheet (AS) provided on this module.

I. Matching Type. (2-Point Each):

1. It generated by the CPU itself to indicate that some error or condition occurred for which assistance from the operating system is needed.

2. It generated by programs when they want to request a system call to be performed by the operating system.

3. It signals sent to the CPU by external devices, normally I/O devices.

4. It uses a table and the interrupt vector to find OS the code to execute in response to interrupts.

5. It is the IRQ for hardware interrupts and an argument to the interrupt assembly language instruction for software interrupts.
- A. CPU

B. Hardware of the CPU

C. User mode

D. Supervisor mode

E. Interrupt vector

F. Hardware Interrupts

G. Traps

H. Software Interrupts

I. Interrupts

6. It generated by hardware devices to signal that they need some attention from the OS.
7. He developed the first theory for computer software in an essay he wrote in 1935.
8. It which is what allows operating systems to take control away from the current running user process.
9. It Can only execute a subset of instructions.
10. It Can execute all machine instructions.

Laboratory Activity

Task 3.1

Instruction: Copy the source code and paste it to your prepared IDE and save it with a file named "first.cpp".

AIM: To write a C program to simulate the CPU scheduling algorithm First Come First Serve (FCFS)

DESCRIPTION: To calculate the average waiting time using the FCFS algorithm first the waiting time of the first process is kept zero and the waiting time of the second process is the burst time of the first process and the waiting time of the third process is the sum of the burst times of the first and the second process and so on. After calculating all the waiting times the average waiting time is calculated as the average of all the waiting times. FCFS mainly says first come first serve the algorithm which came first will be served first.

ALGORITHM:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue

Step 3: For each process in the ready Q, assign the process name and the burst time Step 4: Set the waiting of the first process as 0 and its burst time as its turnaround time Step 5: for each process in the Ready Q calculate

- a). $\text{Waiting time (n)} = \text{waiting time (n-1)} + \text{Burst time (n)}$ b). $\text{Turnaround time (n)} = \text{waiting time(n)} + \text{Burst time(n)}$

Step 6: Calculate

- a) $\text{Average waiting time} = \text{Total waiting Time} / \text{Number of process}$
b) $\text{Average Turnaround time} = \text{Total Turnaround Time} / \text{Number of process}$ Step 7: Stop the process

Source Code:

```
#include <iostream>
int main() {
    int bt[20], wt[20], tat[20], i, n;
    float wtavg, tatavg;
    printf("\nEnter the number of processes -- ");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        printf("\nEnter Burst Time for Process %d -- ", i);
        scanf("%d", &bt[i]);
    }
    wt[0] = wtavg = 0;
    tat[0] = tatavg = bt[0];
    for(i=1; i<n; i++)
    {
        wt[i] = wt[i-1] + bt[i-1];
```

```
tat[i] = tat[i-1] +bt[i];
wtavg = wtavg + wt[i];
tatavg = tatavg + tat[i];
}
printf("\t PROCESS \tBURST TIME \t WAITING TIME\t TURNAROUND TIME\n");
for(i=0;i<n;i++)
printf("\n\t P%d \t\t %d \t\t %d \t\t %d", i, bt[i], wt[i], tat[i]);
printf("\nAverage Waiting Time -- %f", wtavg/n);
printf("\nAverage Turnaround Time -- %f", tatavg/n);

return 0;
}
```

Output:

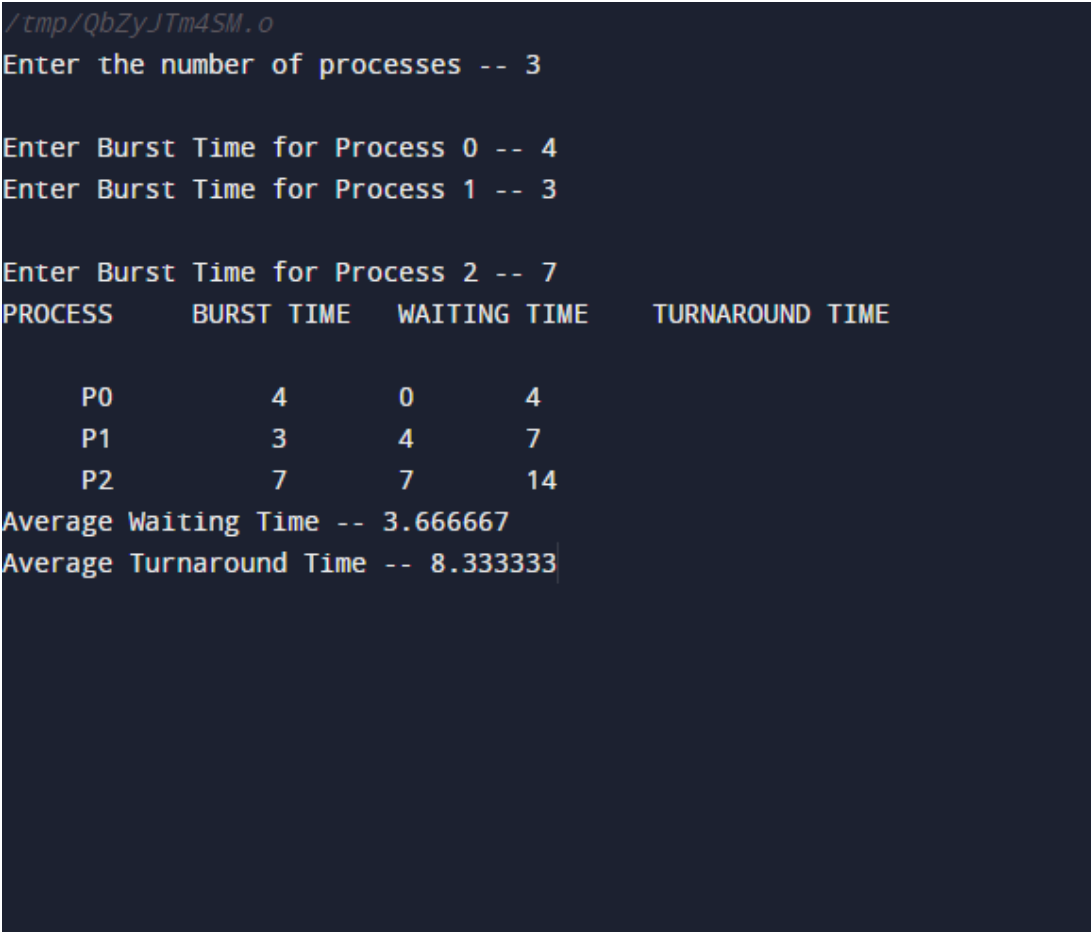


Figure 3.2.

References:

- 1. **Programming Language Definition–**
<https://trustonailende.com/programming-languages/>
- 2. **Lecture Notes on Generation and Types of Programming Language–**
<https://www.includehelp.com/basics/generations-of-programming-language.aspx>
<https://www.typesnuses.com/types-of-programming-languages-with-differences/>

Facilitated By:		
Name	:	
MS Teams Account (email)	:	
Facebook Page	:	